Python for HPC

SDSC Summer Institute 2026
Andrea Zonca
Friday, August 7 | 8:30 AM to 11:20 AM

SAN DIEGO SUPERCOMPUTER CENTER

UC San Diego
HALICIOĞLU SCHOOL OF DATA SCIENCE AND COMPUTING

What you will be able to do

- Compile and benchmark a numerical loop with Numba.
- Choose threads or processes from workload evidence.
- Choose Dask chunks and scale one expression across nodes.

How today works

- Core path: Numba, scheduler choice, and a Dask capstone.
- Deep dives are optional and ready for later.
- Blue note means stuck. Yellow means ready.
- Questions are welcome at every transition.

Start with evidence

- Verify the answer.
- Time or profile a baseline.
- Change one layer at a time.
- Verify again and compare steady-state cost.

Numba compiles the hot loop

- @njit turns supported Python into machine code.
- The first call compiles a signature.
- Warm up before timing.
- Best target: a numerical loop that does not vectorize cleanly.

Hands-on 1: benchmark fairly

- Complete sum_of_squares.
- Check the result against NumPy.
- Warm up the compiled function.
- Time both versions and explain the evidence.
- 12 minutes. Blue means help. Yellow means ready.

Threads and processes

- Threads share memory and one interpreter.
- The GIL limits simultaneous pure-Python bytecode.
- Processes use separate interpreters and memory.
- Startup and serialization are real costs.

Hands-on 2: predict, then check

- Predict CPU-bound pure Python.
- Predict waiting work.
- Run both schedulers with four workers.
- Explain any result that surprises you.
- 12 minutes. Discuss with a neighbor.

Dask separates graph from execution

- Delayed calls build tasks instead of running immediately.
- Dependencies form a directed acyclic graph.
- The scheduler decides when and where tasks run.
- compute() triggers execution.

Chunks are the unit of array work

- Too small: scheduling overhead dominates.
- Too large: memory pressure and weak parallelism.
- Useful chunks fit memory and keep workers busy.
- Inspect chunks, blocks, and bytes before compute().

From one node to multiple nodes

- Notebook builds the graph and requests a result.
- Scheduler tracks dependencies and assigns tasks.
- Workers hold chunks and execute tasks.
- The Dask array expression stays the same.

Capstone: run the cluster

- Start the scheduler on the Jupyter node.
- Submit the worker job from a login terminal.
- Verify workers on distinct nodes.
- Compute, inspect evidence, then cancel the job.
- 18 minutes. Clean up before moving on.

AI is a hypothesis generator

- Provide environment, limits, and correctness criteria.
- Ask for one small change and an explanation.
- Inspect commands and resource requests.
- Test correctness before performance.
- Keep only evidence-backed improvements.

AI exercise: ask, inspect, test

- Ask for one Numba optimization.
- Find package and resource assumptions.
- Check that timing excludes compilation.
- Run correctness and benchmark tests.
- Accept, revise, or reject the suggestion.

Choose the smallest useful layer

- Hot numerical loop: Numba.
- Waiting tasks: threads or delayed.
- CPU-bound Python: processes.
- Chunked array: Dask array.
- More than one node: distributed scheduler.

What you take home

- Tested core and optional notebooks.
- Production SI26 SLURM and Galyleo templates.
- A debug-queue validation workflow.
- An AI review checklist.
- One decision map for your next slow program.